Neural Networks from Scratch:
A Beginner's Guide to PyTorch

Regression and Classification

For Beginners in Artificial Neural Networks

March 23, 2026

Contents

1 Introduction: What Are Neural Networks?

Imagine teaching a computer to recognize handwritten digits or predict house prices without explicitly programming rules. **Neural networks** are computational systems inspired by the human brain that learn patterns from data.

1.1 The Big Picture

Key Idea

A neural network is a universal function approximator. Given enough data and proper training, it can learn to map inputs to outputs for almost any problem:

- **Regression**: Predicting continuous values (house prices, temperature)
- **Classification**: Predicting categories (digit recognition, spam detection)

1.2 Why PyTorch?

PyTorch is a Python library that provides:

1. **Tensors**: Multi-dimensional arrays (like NumPy) but with GPU acceleration
2. **Automatic Differentiation**: Computes gradients automatically (backpropagation)
3. **Neural Network Modules**: Pre-built layers and training utilities

2 Part 1: Regression — Predicting House Prices

We start with **regression**: predicting a continuous number (California housing prices) from input features.

2.1 Step 1: Setup and Data Loading

```
import torch
import torch.nn as nn
import torch.optim as optim
import time
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset
```

What each import does:

- `torch.nn`: Neural network layers (Linear, ReLU, etc.)
- `torch.optim`: Optimization algorithms (SGD, Adam)
- `fetch_california_housing`: Built-in dataset (20,640 houses, 8 features each)
- `train_test_split`: Splits data into training/validation/test sets
- `StandardScaler`: Normalizes features (critical for neural networks!)

2.2 Step 2: Device Configuration

```
# Use GPU for processing if available, otherwise CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

GPU vs CPU

- **GPU** (Graphics Processing Unit): Specialized for parallel math operations. Training neural networks is 10-100x faster on GPU.
- **CPU**: General purpose processor. Use when GPU unavailable.
- `.to(device)` moves tensors/models to the chosen device.

2.3 Step 3: Load and Explore the Dataset

```
# Load California Housing dataset
data = fetch_california_housing()
X = data.data      # Features: (20640, 8) - 20,640 houses, 8 features each
y = data.target    # Target: (20640,) - median house value for each

print("Features:", X.shape)   # Output: (20640, 8)
print("Target:", y.shape)     # Output: (20640,)
```

The 8 Features:

1. Median income in block group
2. House age
3. Average rooms per household
4. Average bedrooms per household
5. Population
6. Average occupancy
7. Latitude
8. Longitude

Visualizing Feature Distributions:

```
plt.figure(figsize=(12,6))
for i in range(6):  # Plot first 6 features
    plt.subplot(2, 3, i+1)
    plt.hist(X[:, i], bins=40)  # Histogram of feature values
    plt.title(data.feature_names[i])
plt.tight_layout()
plt.show()
```

This shows if features have different scales (which they do!), indicating we need **standardization**.

2.4 Step 4: Train-Validation-Test Split

```
# First split: 70% train, 30% temp (will become val + test)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Second split: Split temp into 50% validation, 50% test
# Final: 70% train, 15% validation, 15% test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

Why Three Sets?

- **Training Set (70%)**: Used to update model weights
- **Validation Set (15%)**: Used to tune hyperparameters and detect overfitting
- **Test Set (15%)**: Used only ONCE at the end to evaluate final performance

Never use the test set during training or hyperparameter tuning!

2.5 Step 5: Feature Standardization (Critical!)

```
# Calculate mean and standard deviation from TRAINING data only!
mean = X_train.mean(axis=0)    # Mean of each feature (8 values)
std = X_train.std(axis=0)      # Std dev of each feature (8 values)

# Add small epsilon to avoid division by zero
eps = 1e-8
std = std + eps

# Standardize: (x - mean) / std
# Result: Features have mean=0, std=1
X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std
```

Common Mistake

Use training statistics (mean, std) to transform validation and test sets. Never calculate separate statistics for each set—this would leak information!

Why Standardize?

- Neural networks train faster when inputs have similar scales
- Prevents features with large ranges (like income) from dominating
- Helps gradients flow better during backpropagation

2.6 Step 6: Convert to PyTorch Tensors

```
# Convert NumPy arrays to PyTorch tensors
# dtype=torch.float32: Neural networks need floating point numbers
# .to(device): Move to GPU if available
```

```
4
5  X_train = torch.tensor(X_train, dtype=torch.float32).to(device)
6  y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1).to(device)
7
8  X_val = torch.tensor(X_val, dtype=torch.float32).to(device)
9  y_val = torch.tensor(y_val, dtype=torch.float32).view(-1, 1).to(device)
10
11 X_test = torch.tensor(X_test, dtype=torch.float32).to(device)
12 y_test = torch.tensor(y_test, dtype=torch.float32).view(-1, 1).to(device)
```

Understanding `.view(-1, 1)`:

- Original `y` shape: `(14448,)` — 1D array
- Neural networks expect 2D input: `(batch_size, features)`
- `.view(-1, 1)` reshapes to `(14448, 1)` — column vector
- `-1` means "infer this dimension automatically"

3 Building the Neural Network Architecture

3.1 The RegressionNet Class

```
class RegressionNet(nn.Module):
    """
    A Neural Network for Regression Tasks (Predicting Continuous Values)

    Architecture: Input -> Hidden -> Hidden -> Output
                  (8)   ->  (64)  ->  (64)   ->  (1)
    """

    def __init__(self, input_dim):
        super().__init__()
        # nn.Sequential: Container that chains layers together
        self.net = nn.Sequential(
            # Layer 1: Linear transformation + activation
            nn.Linear(input_dim, 64),  # 8 -> 64
            nn.ReLU(),                 # Activation

            # Layer 2: Hidden layer
            nn.Linear(64, 64),         # 64 -> 64
            nn.ReLU(),

            # Layer 3: Output layer (no activation!)
            nn.Linear(64, 1)           # 64 -> 1 (predicted price)
        )

    def forward(self, x):
        """Define the forward pass: input -> output"""
        return self.net(x)
```

3.2 Understanding Each Component

Layer-by-Layer Breakdown

Layer 1: `nn.Linear(input_dim, 64)`

- Mathematical operation: $\mathbf{y} = \mathbf{x}\mathbf{W}^T + \mathbf{b}$
- Input: $(\text{batch\_size}, 8)$
- Output: $(\text{batch\_size}, 64)$
- Parameters: 64×8 weights $+ 64$ biases $= 576$ numbers to learn

Activation: `nn.ReLU()`

- Function: $f(x) = \max(0, x)$
- Purpose: Introduces non-linearity
- Without activation, stacking linear layers = single linear layer!
- Example: $[-2, 5, -1, 3] \rightarrow [0, 5, 0, 3]$

Layer 3: Output

- No activation function for regression
- We want raw numbers (house prices can be any positive value)
- ReLU would force positive only; sigmoid would squash to $[0, 1]$

4 Training Methods: Three Types of Gradient Descent

4.1 Method 1: Batch Gradient Descent

```
def train_batch_gd(X_train, y_train, X_val, y_val, epochs=10, lr=0.1):
    """
    BATCH GRADIENT DESCENT: Uses ENTIRE dataset for each update.
    Most stable but memory-intensive for large datasets.
    """
    model = RegressionNet(X_train.shape[1]).to(device)
    criterion = nn.MSELoss()           # Mean Squared Error for regression
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_losses, val_losses = [], []
    start = time.time()

    for epoch in range(epochs):
        model.train()                  # Training mode
        optimizer.zero_grad()          # Clear old gradients

        # FORWARD: Process ALL training data at once
        preds = model(X_train)         # Shape: (14448, 1)
        train_loss = criterion(preds, y_train)

        # BACKWARD: Compute gradients
        train_loss.backward()

        # UPDATE: Adjust weights
```

```
        optimizer.step()

        # VALIDATION: Check generalization
        model.eval()
        with torch.no_grad():
            val_loss = criterion(model(X_val), y_val)

        train_losses.append(train_loss.item())
        val_losses.append(val_loss.item())

        print(f"Epoch {epoch:02d} | Train: {train_loss.item():.4f} | Val: {
    val_loss.item():.4f}")

    return train_losses, val_losses, time.time() - start, model
```

The Training Loop Explained

1. **Forward Pass**: Compute predictions for all data
2. **Compute Loss**: How wrong are the predictions?
3. **Backward Pass**: Calculate gradients (which way to adjust weights)
4. **Update Weights**: Take a step in the direction that reduces loss
5. **Validate**: Check performance on unseen data (no training!)

4.2 Method 2: Stochastic Gradient Descent (SGD)

```
def train_sgd(X_train, y_train, X_val, y_val, epochs=5, lr=0.001):
    """
    STOCHASTIC GRADIENT DESCENT: Uses ONE random sample per update.
    "Stochastic" = random. Noisy but often escapes local minima.
    """
    model = RegressionNet(X_train.shape[1]).to(device)
    criterion = nn.MSELoss()
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_losses, val_losses = [], []
    N = len(X_train)
    start = time.time()

    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(N)    # Random shuffle indices

        for idx, i in enumerate(perm):
            optimizer.zero_grad()

            # Process SINGLE sample
            preds = model(X_train[i:i+1])      # Keep 2D: (1, 8)
            loss = criterion(preds, y_train[i:i+1])

            loss.backward()
            optimizer.step()

            train_losses.append(loss.item())

            if (idx + 1) % 5000 == 0:
                print(f"Step {idx+1} | Loss: {loss.item():.4f}")

```

```
        # End-of-epoch validation
        model.eval()
        with torch.no_grad():
            train_loss = criterion(model(X_train), y_train)
            val_loss = criterion(model(X_val), y_val)
        val_losses.append(val_loss.item())

    return train_losses, val_losses, time.time() - start, model
```

Key Differences from Batch GD:

- Updates weights after every single sample (not after all)
- Requires shuffling each epoch (randomness is crucial)
- Lower learning rate (0.001 vs 0.1) because single samples are noisy
- `X_train[i:i+1]` keeps tensor 2D; `X_train[i]` would make it 1D

4.3 Method 3: Mini-Batch Gradient Descent (Recommended)

```
def train_minibatch_gd(X_train, y_train, X_val, y_val,
                       batch_size=16, epochs=10, lr=0.01):
    """
    MINI-BATCH GRADIENT DESCENT: Uses small groups (batches) of samples.
    Best of both worlds: stable like batch GD, fast like SGD.
    """
    model = RegressionNet(X_train.shape[1]).to(device)
    criterion = nn.MSELoss()
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_losses, val_losses = [], []
    N = len(X_train)
    start = time.time()

    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(N)
        X_shuffled = X_train[perm]
        y_shuffled = y_train[perm]

        epoch_batch_losses = []

        # Process data in chunks of batch_size
        for batch_start in range(0, N, batch_size):
            X_batch = X_shuffled[batch_start : batch_start + batch_size]
            y_batch = y_shuffled[batch_start : batch_start + batch_size]

            optimizer.zero_grad()

            # Forward pass on batch
            preds = model(X_batch)
            loss = criterion(preds, y_batch)

            # Backward and update
            loss.backward()
            optimizer.step()

            epoch_batch_losses.append(loss.item())

        train_losses.extend(epoch_batch_losses)

```

```
        # Validation
        model.eval()
        with torch.no_grad():
            train_loss = criterion(model(X_train), y_train)
            val_loss = criterion(model(X_val), y_val)
        val_losses.append(val_loss.item())

    return train_losses, val_losses, time.time() - start, model
```

Why Mini-Batch is Best

- **Vectorization**: GPUs process batches efficiently (parallel computation)
- **Stability**: Averaged over 16 samples vs single sample noise
- **Memory**: Doesn't require loading entire dataset at once
- **Common batch sizes**: 16, 32, 64, 128, 256

4.4 Comparison of Three Methods

Aspect	Batch GD	SGD	Mini-Batch
Samples per update	All (14,448)	1	16 (configurable)
Updates per epoch	1	14,448	903
Speed per update	Slow	Fast	Medium
Gradient noise	None	High	Low
Memory usage	High	Low	Medium
Typical use case	Small datasets	Online learning	Standard practice

Table 1: Comparison of Gradient Descent Variants

5 Visualizing Training Progress

5.1 Converting Tensors for Plotting

```
def to_numpy(x):
    """
    Convert list of PyTorch tensors to NumPy array for matplotlib.
    Need to: 1) Detach from computation graph, 2) Move to CPU, 3) Extract value
    """
    return np.array([
        v.detach().cpu().item() if torch.is_tensor(v) else v
        for v in x
    ])

# Convert training history
mb_tr = to_numpy(mb_tr)      # Training losses (per batch)
mb_val = to_numpy(mb_val)    # Validation losses (per epoch)
```

Why these operations?

- `.detach()`: Remove gradient tracking (saves memory)
- `.cpu()`: Move from GPU to CPU memory (matplotlib needs CPU)
- `.item()`: Extract Python number from single-value tensor

5.2 Aligning Different Sampling Rates

Training loss is recorded per batch; validation loss per epoch. To plot together:

```
# Create x-coordinates
x_train = np.arange(len(mb_tr))  # [0, 1, 2, ..., 902] for 903 batches

# Spread validation points evenly across training timeline
x_val = np.linspace(0, len(mb_tr) - 1, num=len(mb_val))

# Interpolate: Connect validation points with straight lines
mb_val_stretched = np.interp(x_train, x_val, mb_val)

# Plot
plt.plot(x_train, mb_tr, label="Train", alpha=0.7)
plt.plot(x_train, mb_val_stretched, label="Validation")
plt.xlabel("Training Steps")
plt.ylabel("MSE Loss")
plt.title("Training Progress")
plt.legend()
plt.show()
```

Reading the Training Curve

- **Both curves decreasing**: Model is learning!
- **Training ↓, Validation ↑**: Overfitting (memorizing training data)
- **Flat lines**: Learning rate too low or model stuck
- **Spiky**: Learning rate too high

6 Part 2: Classification — Handwritten Digit Recognition

Now we switch to **classification**: predicting discrete categories (digits 0-9).

6.1 Loading MNIST Dataset

```
from torchvision.datasets import MNIST
from torchvision.transforms import ToTensor

# Download and load MNIST dataset
train_data = MNIST(root=".", train=True, download=True, transform=ToTensor())
test_data = MNIST(root=".", train=False, download=True, transform=ToTensor())
```

MNIST Dataset:

- 70,000 grayscale images of handwritten digits (0-9)
- Each image: 28×28 pixels
- Training set: 60,000 images
- Test set: 10,000 images

6.2 Data Preparation

```python
def prepare(dataset):
    """Flatten 28x28 images to 784-dimensional vectors and normalize."""
    # view(-1, 784): Reshape (N, 28, 28) -> (N, 784)
    # / 255.0: Normalize pixel values from [0, 255] to [0, 1]
    X = dataset.data.view(len(dataset), -1).float() / 255.0
    y = dataset.targets
    return X, y

X, y = prepare(train_data)      # Training data
X_test, y_test = prepare(test_data)  # Test data

# Split training into train (90%) and validation (10%)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.1, random_state=42
)
```

6.3 Creating DataLoaders

```python
# Wrap in TensorDataset for easy indexing
train_dataset = TensorDataset(X_train, y_train)
val_dataset = TensorDataset(X_val, y_val)

# DataLoader handles batching and shuffling automatically
train_loader = DataLoader(
    train_dataset,
    batch_size=32,       # Process 32 images at once
    shuffle=True,        # Randomize order each epoch
    drop_last=False      # Keep last incomplete batch
)

val_loader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False        # No need to shuffle validation
)
```

DataLoader Benefits

- Automatic batching
- Multi-processing for fast data loading
- Shuffling each epoch (prevents memorization)
- Handles last incomplete batch automatically

6.4 One-Hot Encoding (Conceptual)

```python
def one_hot(labels, num_classes=10):
    """Convert class indices to one-hot vectors."""
    # torch.eye(10) creates 10x10 identity matrix
    # Indexing with labels selects corresponding rows
    return torch.eye(num_classes)[labels]

# Example: digit 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
y_train_oh = one_hot(y_train)
```

Note: Modern PyTorch uses `CrossEntropyLoss` which accepts raw class indices, so explicit one-hot encoding is no longer necessary.

6.5 Classification Network Architecture

```python
class MNISTNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 128),    # Input: 784 pixels -> 128 features
            nn.ReLU(),
            nn.Linear(128, 64),     # 128 -> 64
            nn.ReLU(),
            nn.Linear(64, 10)       # 64 -> 10 (one per digit class)
            # NO activation! CrossEntropyLoss includes Softmax internally
        )

    def forward(self, x):
        return self.net(x)
```

Differences from Regression:

- Output dimension = number of classes (10 for digits 0-9)
- No final activation (Softmax is inside CrossEntropyLoss)
- Output represents **logits** (unnormalized scores)

6.6 Training Function for Classification

```python
def train_model(model, optimizer, epochs=10, batch_size=32):
    """
    Train a classification model.
    Uses CrossEntropyLoss and tracks accuracy.
    """
    criterion = nn.CrossEntropyLoss()

    # Create fresh dataloaders
    train_loader = DataLoader(
        TensorDataset(X_train, y_train),
        batch_size=batch_size, shuffle=True
    )
    val_loader = DataLoader(
        TensorDataset(X_val, y_val),
        batch_size=batch_size, shuffle=False
    )

    val_acc = []

    for epoch in range(epochs):
        # ----- TRAINING -----
        model.train()
        running_loss = 0.0

        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)

            optimizer.zero_grad()
            outputs = model(xb)           # Shape: (32, 10) - logits for each class
            loss = criterion(outputs, yb)   # yb: class indices (0-9)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
```

```
        avg_loss = running_loss / len(train_loader)

        # ----- VALIDATION -----
        model.eval()
        correct, total = 0, 0

        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device), yb.to(device)
                outputs = model(xb)

                # Get predicted class (index of maximum logit)
                preds = torch.argmax(outputs, dim=1)

                correct += (preds == yb).sum().item()
                total += yb.size(0)

        acc = correct / total
        val_acc.append(acc)

        print(f"Epoch [{epoch+1}/{epochs}] | Loss: {avg_loss:.4f} | Acc: {acc
    :.4f}")

    return val_acc
```

Classification vs Regression Differences

Aspect	Regression	Classification
Output	1 value	10 class scores (logits)
Loss Function	MSELoss	CrossEntropyLoss
Target Format	Float values	Integer class indices (0-9)
Metric	MSE (lower better)	Accuracy (higher better)
Final Activation	None	Softmax (inside loss)

7 Optimizer Comparison Experiment

7.1 Testing Different Optimizers

```
histories = {}

# Experiment 1: SGD with Momentum
print("\nTraining with SGD (lr=0.1, momentum=0.9)")
model = MNISTNet().to(device)
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.1,
    momentum=0.9,        # Accelerates in consistent directions
    weight_decay=1e-4    # L2 regularization
)
histories["SGD"] = train_model(model, optimizer)

# Experiment 2: Adagrad (adaptive learning rates)
print("\nTraining with Adagrad (lr=0.01)")
model = MNISTNet().to(device)
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01, weight_decay=1e-4)
histories["Adagrad"] = train_model(model, optimizer)
```

```python

# Experiment 3: RMSProp (fixes Adagrad's decay)
print("\nTraining with RMSProp (lr=0.001)")
model = MNISTNet().to(device)
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.001, alpha=0.99)
histories["RMSProp"] = train_model(model, optimizer)

# Experiment 4: Adam (most popular, adaptive + momentum)
print("\nTraining with Adam (lr=0.001)")
model = MNISTNet().to(device)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    betas=(0.9, 0.999),  # (momentum decay, squared grad decay)
    weight_decay=1e-4
)
histories["Adam"] = train_model(model, optimizer)
```

7.2 Visualizing Optimizer Comparison

```python
# Plot all optimizer histories
for name, acc in histories.items():
    plt.plot(acc, label=name)

plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.title("Optimizer Comparison on MNIST")
plt.legend()
plt.show()
```

Optimizer Characteristics

- **SGD + Momentum**: Classic, needs learning rate tuning, can escape sharp minima
- **Adagrad**: Good for sparse data, learning rate decreases over time (can stop too early)
- **RMSProp**: Fixes Adagrad's aggressive decay, good for RNNs
- **Adam**: Combines momentum + adaptive rates, works well out-of-the-box, most popular

7.3 Learning Rate Experiment

```python
# Test different learning rates with Adam
lrs = [0.0001, 0.001, 0.01]

for lr in lrs:
    print(f"\nTraining with Adam (lr={lr})")
    model = MNISTNet().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    acc = train_model(model, optimizer)
    plt.plot(acc, label=f"LR={lr}")

plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.title("Learning Rate Effect (Adam)")
plt.legend()
plt.show()
```

Learning Rate Matters

- **Too high (0.01)**: Unstable training, loss spikes, may diverge
- **Just right (0.001)**: Smooth convergence, good final performance
- **Too low (0.0001)**: Slow convergence, may get stuck in local minima

8 Key Takeaways and Best Practices

8.1 Checklist for Training Neural Networks

1. **Preprocessing**
   - Split data: Train/Val/Test (typically 70/15/15 or 80/10/10)
   - Standardize features: Zero mean, unit variance
   - Use training statistics for validation/test transforms
2. **Architecture**
   - Input dimension = number of features
   - Hidden layers: Start with 1-2 layers, 64-128 neurons
   - ReLU activation between hidden layers
   - Output: 1 neuron (regression) or N neurons (classification)
3. **Training**
   - Use mini-batch gradient descent (batch size 32-256)
   - Shuffle training data every epoch
   - Start with Adam optimizer (lr=0.001)
   - Monitor validation loss to detect overfitting
4. **Evaluation**
   - Use separate test set (never seen during training)
   - Classification: Report accuracy, confusion matrix
   - Regression: Report MSE, MAE, R^2 score

8.2 Common Pitfalls

Mistakes to Avoid

1. **Not calling .zero_grad()**: Gradients accumulate by default!
2. **Forgetting model.eval()**: Dropout/BatchNorm behave differently in eval mode
3. **Not using torch.no_grad() for inference**: Wastes memory computing unnecessary gradients
4. **Data leakage**: Using test set statistics for standardization
5. **Wrong loss function**: MSE for classification or CrossEntropy for regression

9 Further Reading

- PyTorch Official Tutorials: `https://pytorch.org/tutorials/`
- "Deep Learning" by Goodfellow, Bengio, and Courville (free online)
- 3Blue1Brown Neural Networks series (YouTube)
- Fast.ai Practical Deep Learning for Coders